

DispatcherServlet & Spring MVC Request Flow — Detailed Notes

1. What is DispatcherServlet?

Definition: DispatcherServlet is the **Front Controller** of the Spring MVC framework. It is a single servlet that receives **all incoming HTTP requests** for a Spring web application and delegates them to the appropriate components (Controllers, View Resolvers, etc.) for processing.

Key Facts:

- It extends `HttpServlet` (specifically via `FrameworkServlet`), so at its core it is still a regular Java Servlet.
- It implements the **Front Controller Design Pattern** — instead of having multiple servlets for different URLs, there is **ONE** servlet that handles everything centrally.
- Package: `org.springframework.web.servlet.DispatcherServlet`
- In **Spring Boot**, it is **auto-configured and auto-registered** with the embedded Tomcat, mapped to `/` by default — no manual setup needed.
- In **traditional Spring MVC**, it is registered manually via `web.xml` or `WebApplicationInitializer`.

Class Hierarchy (simplified):

```
HttpServlet
  ↑
GenericServlet
  ↑
HttpServletBean
  ↑
FrameworkServlet (loads Spring's WebApplicationContext)
  ↑
DispatcherServlet (adds MVC-specific request handling)
```

2. Role of DispatcherServlet

Role	Description
Single Entry Point	All requests (<code>/users</code> , <code>/orders</code> , <code>/products</code> , etc.) pass through this one servlet
Request Delegation	Finds and delegates the request to the correct Controller method
Coordination	Coordinates between HandlerMapping, Controller, ViewResolver, and Exception handling
Response Preparation	Converts Controller's return value into the final

Role	Description
	HTTP response (JSON/View)
Centralized Config	Cross-cutting concerns (interceptors, exception handling, data binding) are managed centrally here

Why "Front Controller"? Without it, every URL would need its own separate Servlet (like plain `@WebServlet`). `DispatcherServlet` removes this need — you just write `@Controller` classes with `@RequestMapping`, and `DispatcherServlet` internally handles all routing.

3. How Spring Boot Registers DispatcherServlet Automatically

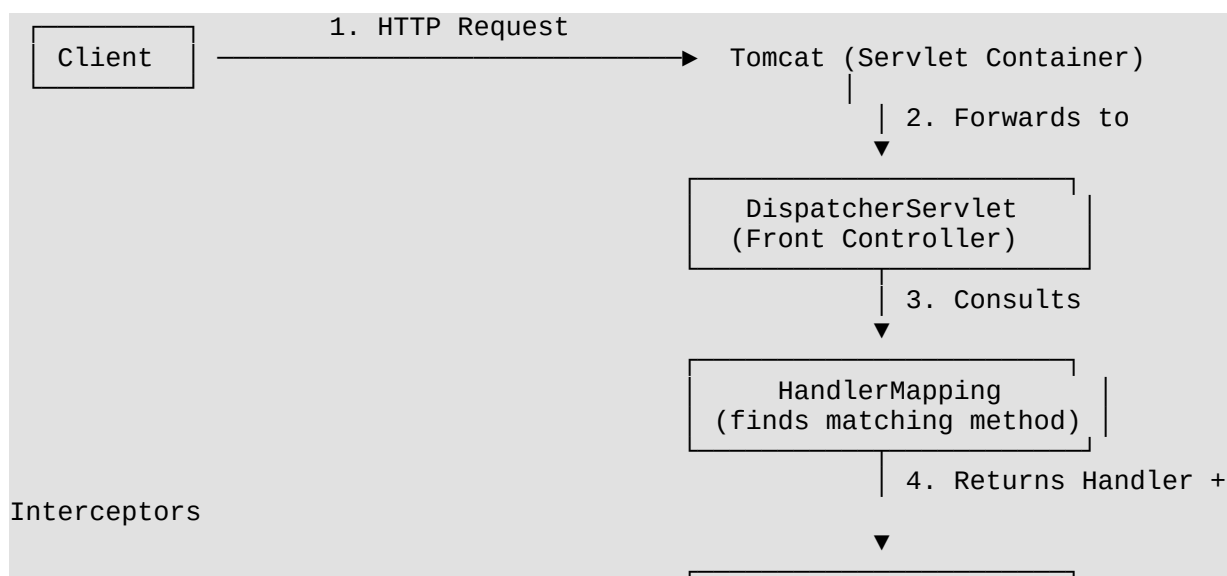
```

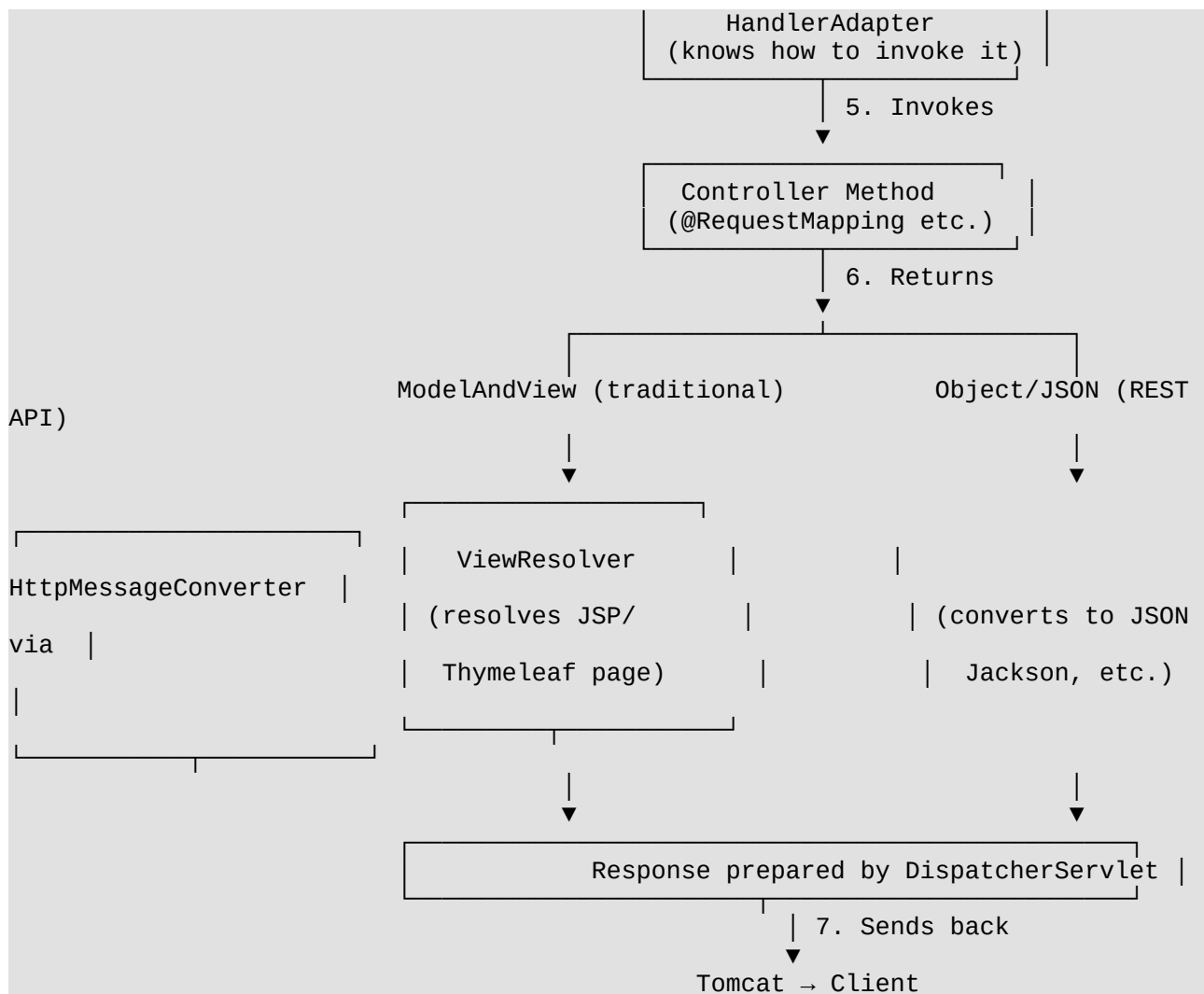
Application starts (main() → SpringApplication.run())
↓
Spring Boot detects spring-webmvc on classpath
↓
DispatcherServletAutoConfiguration is triggered
↓
DispatcherServlet bean is created
↓
Registered via ServletRegistrationBean, mapped to "/"
↓
Embedded Tomcat (TomcatServletWebServerFactory) starts
↓
DispatcherServlet is registered into Tomcat's ServletContext
↓
Tomcat starts listening on port (default: 8080)

```

Because of this, **only one servlet** (`DispatcherServlet`) is ever registered with the container — everything else is handled internally by Spring's MVC machinery.

4. Full Spring MVC Request Flow (Step-by-Step)





5. Detailed Explanation of Each Step

Step 1: Client sends HTTP Request

- Browser/Postman/frontend app sends a request to a URL, e.g., `http://localhost:8080/users`

Step 2: Tomcat receives the request

- The **Connector** in Tomcat accepts the raw TCP/HTTP request and creates `HttpServletRequest` & `HttpServletResponse` objects.
- Since `DispatcherServlet` is mapped to `/`, Tomcat forwards **every** request to it (unless a more specific servlet/filter intercepts first).

Step 3: DispatcherServlet consults HandlerMapping

- `HandlerMapping` is responsible for finding **which Controller method** should handle this specific URL + HTTP method combination.
- It scans all `@Controller/@RestController` classes and matches based on `@RequestMapping`, `@GetMapping`, `@PostMapping`, etc.

- Returns a `HandlerExecutionChain` — the target method plus any `HandlerInterceptors` (e.g., logging, auth checks) that should run before/after.

Step 4: `HandlerAdapter` invokes the Controller

- `HandlerAdapter` acts as a bridge — it knows **how** to actually call the handler method (since handler methods can have very different signatures: different parameters, return types, annotations like `@RequestBody`, `@PathVariable`, etc.)
- It performs **data binding** — converting request parameters, JSON body, path variables into Java method arguments automatically.

Step 5: Controller executes business logic

- The actual `@Controller/@RestController` method runs.
- It typically talks to the **Service layer** → which talks to the **Repository/DAO layer** → which talks to the **Database**.
- Returns either:
 - A `ModelAndView` (data + view name) — for traditional server-rendered apps (JSP/Thymeleaf)
 - A plain object/JSON — for REST APIs (when `@RestController` or `@ResponseBody` is used)

Step 6: Response is prepared

- **If traditional MVC:** `DispatcherServlet` passes the view name to `ViewResolver`, which locates the actual JSP/Thymeleaf file. The `Model` data is used to render that view into final HTML.
- **If REST API:** `HttpMessageConverter` (like `MappingJackson2HttpMessageConverter`) converts the returned Java object directly into JSON (or XML) — no view is involved.

Step 7: Response sent back

- `DispatcherServlet` writes the final response into `HttpServletResponse`.
- Tomcat sends this back over the network to the client.

6. How `DispatcherServlet` "Intercepts" Requests

- It doesn't literally "intercept" in a sneaky sense — it is simply **registered as the servlet mapped to /** (or a specific pattern) in Tomcat's servlet mapping table.
- Because of this mapping, Tomcat's **Mapper** component directs **all matching requests** to it first — exactly like any servlet, just with a broader URL pattern (/ = catches everything not otherwise mapped).
- Before reaching Controllers, requests can also pass through:
 - **Filters** (`javax.servlet.Filter`) — run at the servlet container level, before

DispatcherServlet (e.g., CharacterEncodingFilter, security filters)

- **Interceptors** (HandlerInterceptor) — run **within** Spring MVC, after DispatcherServlet but before/after the Controller method (preHandle(), postHandle(), afterCompletion())

Request → Filters (Servlet-level) → DispatcherServlet → Interceptors (preHandle)
→ Controller → Interceptors (postHandle) → View/Response → Interceptors (afterCompletion)

7. Key Components DispatcherServlet Works With

Component	Purpose
HandlerMapping	Maps incoming request URL to the correct Controller method
HandlerAdapter	Knows how to invoke the matched handler method, handles data binding
HandlerInterceptor	Pre/post processing logic around Controller execution (like filters, but Spring-aware)
ViewResolver	Resolves logical view names (e.g., "home") to actual view files (home.jsp)
HttpMessageConverter	Converts Java objects ↔ JSON/XML for REST APIs
HandlerExceptionResolver	Handles exceptions thrown during request processing (works with @ExceptionHandler, @ControllerAdvice)
LocaleResolver	Determines locale (for internationalization)
MultipartResolver	Handles file upload requests (multipart/form-data)

8. Quick Summary (for revision)

- **DispatcherServlet** = Spring MVC's **Front Controller** — a single servlet that receives all requests.
- It is auto-registered by Spring Boot (mapped to /) using auto-configuration + embedded Tomcat.
- **Flow:** Client → Tomcat → DispatcherServlet → HandlerMapping → HandlerAdapter → Controller → (ViewResolver / HttpMessageConverter) → Response → Client
- It "intercepts" requests simply by being the servlet mapped to /, so Tomcat routes matching

requests to it first.

- It works closely with **HandlerMapping**, **HandlerAdapter**, **ViewResolver**, **HttpMessageConverter**, and **HandlerInterceptor** to fully process a request.
- **Benefit:** Centralizes request handling — developers only write Controllers; DispatcherServlet manages all routing/coordination internally.